Informe Técnico — Práctica 1: Bases de Datos I

Diseño, Normalización y Migración del Dataset MotoGP (2000–2021)

Asignatura: Bases de Datos I — Curso 2025/2026
Dataset fuente: `moto_results.csv`
Entregables: `eda.ipynb`, `motogp.sql`, `motogp_mysql.sql`
Herramientas: Python 3.13, pandas, sqlite3, MySQL Workbench

1. Resumen Ejecutivo

El presente informe documenta el proceso íntegro de análisis, diseño e implementación llevado a cabo en el marco de la Práctica 1 de la asignatura Bases de Datos I. El objetivo principal consistió en transformar un fichero plano con datos históricos del Campeonato del Mundo de Motociclismo en un esquema relacional normalizado, implementado sobre MySQL.

El dataset de partida, `moto_results.csv`, contenía 29.931 registros organizados en 17 columnas sin normalización alguna. Tras un análisis exploratorio exhaustivo en el cuaderno `eda.ipynb`, la identificación de dependencias funcionales y la resolución de las anomalías detectadas, se diseñó un esquema compuesto por **seis tablas relacionales** que satisface la Tercera Forma Normal (3FN), elimina redundancias y garantiza la integridad referencial. El proceso culminó con la generación de un script MySQL (`motogp_mysql.sql`) y la validación del modelo mediante seis consultas analíticas recogidas en `consultas_motogp.sql`.

2. Descripción del Dataset Original

2.1 Estructura y tipos de datos

El fichero `moto_results.csv` fue cargado mediante `pandas.read_csv()`. A continuación se aplicaron conversiones de tipo con `df.convert_dtypes()` y `pd.to_datetime()` sobre la columna `date`, obteniendo el esquema de tipos definitivo:

Columna	Tipo original	Tipo convertido
`year`	int64	Int64
`sequence`	int64	Int64
`category`	object	string
`rider_first_name`	object	string
`rider_last_name`	object	string
`rider_number`	float64	Int64

Columna	Tipo original	Tipo convertido
`rider_country`	object	string
`team_name`	object	string
`bike`	object	string
`position`	int64	Int64
`points`	int64	Int64
`speed`	float64	Float64
`time`	object	string
`race_name`	object	string
`circuit_name`	object	string
`circuit_country`	object	string
`date`	object	datetime64[ns]

La estructura original presentaba **redundancia total**: cada fila combinaba atributos pertenecientes a entidades distintas —piloto, equipo, circuito, gran premio y resultado— en una única tupla plana. Esta disposición constituye una violación directa de la 3FN al existir dependencias transitivas y parciales respecto a la clave.

2.2 Estadísticas de cobertura

```
Filas totales:     29.931
Columnas:          17
Rango temporal:    2000 - 2021
Categorías:        7
  MotoGP   →  7.023 registros
  Moto2    →  6.755 registros
  125cc    →  6.013 registros
  Moto3    →  5.518 registros
  250cc    →  3.856 registros
  500cc    →    502 registros
  MotoE    →    264 registros
Equipos distintos: 970
```

3. Auditoría de Datos (eda.ipynb — Paso 1)

3.1 Análisis de valores nulos

La inspección mediante `df.info()` y `df.isnull().sum()` identificó tres columnas con valores ausentes:

Columna	Registros nulos	Causa identificada
`rider_number`	5.127	Ausencia de dorsal en registros históricos (125cc, 250cc, 500cc)
`speed`	992	Pilotos que no tomaron la salida o fueron descalificados antes de completar una vuelta
`time`	1	Registro aislado sin dato de tiempo

Se constató una correlación exacta entre los 5.127 registros con `rider_number` nulo y los 5.127 registros con `team_name = '?'`, verificada mediante comparación de máscaras booleanas:

```
mask_team   = df['team_name'] == '?'
mask_number = df['rider_number'].isnull()
assert (mask_team == mask_number).all()  # True
```

Esta coincidencia indica que los registros históricos sin dorsal conocido corresponden sistemáticamente a participaciones sin equipo identificado en la fuente original. Antes de la migración, el token `'?'` fue sustituido por `None` mediante:

```
df['team_name'] = df['team_name'].replace('?', None)
```

3.2 Normalización de circuit_name

Se detectaron inconsistencias de formato en la columna `circuit_name` que generaban duplicados lógicos del mismo circuito. Se aplicaron las siguientes correcciones:

```
# Corrección de país incorrecto
df.loc[df["circuit_name"] == "MotorLand Aragón", "circuit_country"] = "ES"

# Normalización de formato
df['circuit_name'] = (
    df['circuit_name']
    .str.strip()    # eliminación de espacios al inicio y final
    .str.title()    # estandarización a mayúscula inicial por palabra
)
```

3.3 Análisis de posiciones especiales

El campo `position` contenía 5.475 valores no positivos, correspondientes al 18,29 % del total. El cuaderno los mapea del siguiente modo:

Código	Significado	Registros
-1	DNF — Did Not Finish	mayoría de los 5.475
-2	DNS — Did Not Start	—
-3	DSQ — Descalificado	—

Código	Significado	Registros
-4	NC — Not Classified	—
-5	Otro	—

Se verificó que la integridad de puntos se mantiene en todos estos casos:

```
assert (df[df['position'] <= 0]['points'] == 0).all()
# Confirmado: ningún registro con posición no positiva tiene puntos asignados
```

3.4 Homogeneización de códigos de país

Durante la fase de validación de consultas se detectó que `riders.nationality` empleaba el estándar **ISO 3166-1 alpha-3** (tres letras: SPA, ITA, GBR), mientras que `circuits.country` utilizaba **alpha-2** (dos letras: ES, IT, GB). Esta discrepancia impedía la comparación directa en la Consulta 6.

La solución se implementó en `motogp.sql` mediante un `UPDATE` sobre la tabla `circuits` que mapea los 20 códigos alpha-2 a su equivalente alpha-3:

```
UPDATE circuits
SET country = CASE country
    WHEN 'AR' THEN 'ARG'    WHEN 'AT' THEN 'AUT'
    WHEN 'AU' THEN 'AUS'    WHEN 'BR' THEN 'BRA'
    WHEN 'CN' THEN 'CHN'    WHEN 'CZ' THEN 'CZE'
    WHEN 'DE' THEN 'GER'    WHEN 'ES' THEN 'SPA'
    WHEN 'FR' THEN 'FRA'    WHEN 'GB' THEN 'GBR'
    WHEN 'IT' THEN 'ITA'    WHEN 'JP' THEN 'JPN'
    WHEN 'MY' THEN 'MAL'    WHEN 'NL' THEN 'NED'
    WHEN 'PT' THEN 'POR'    WHEN 'QA' THEN 'QAT'
    WHEN 'TH' THEN 'THA'    WHEN 'TR' THEN 'TUR'
    WHEN 'US' THEN 'USA'    WHEN 'ZA' THEN 'RSA'
    ELSE country
END
WHERE id_circuit >= 1;
```

4. Análisis de Dependencias Funcionales (eda.ipynb — Paso 2)

El análisis de dependencias funcionales constituye el fundamento del diseño relacional. A continuación se documentan las comprobaciones realizadas y las conclusiones derivadas de su ejecución en el cuaderno.

4.1 Idoneidad del dorsal como identificador de piloto

```
Coincidencias encontradas: 14

Año 2006 Cat 125cc Dorsal 80  → Doni Tata Pradita, Tito Rabat
Año 2006 Cat MotoGP Dorsal 8  → Garry Mccoy, Naoki Matsudo
Año 2007 Cat 125cc Dorsal 51  → Stevie Bonsey, Steve Bonsey
```

```
Año 2009 Cat 125cc Dorsal 76  → Ivan Maestro, Toni Finsterbusch
Año 2010 Cat 125cc Dorsal 57  → Joel Taylor, Isaac Viales
```

Conclusión: `rider_number` no puede actuar como clave de la entidad PILOTO. El dorsal depende del contexto competitivo (año y categoría), por lo que se ubica en RESULTADO con valor NULL permitido.

4.2 Estabilidad de la nacionalidad del piloto

```
Pilotos con más de un país registrado: 0
```

Conclusión: `rider_country` es un atributo propio de la entidad PILOTO, con dependencia funcional plena respecto al identificador del piloto.

4.3 Estabilidad del país del circuito

```
Circuitos con más de un país registrado: 0  (tras la normalización del §3.2)
```

Conclusión: `circuit_country` pertenece exclusivamente a la entidad CIRCUITO.

4.4 Fiabilidad del nombre de carrera como identificador

```
Carreras con más de un circuito: 1
"Japanese Grand Prix" → Twin Ring Motegi y Suzuka Circuit
```

Conclusión: `race_name` no constituye un identificador fiable. Se conserva únicamente como atributo descriptivo en la entidad GRAN PREMIO.

4.5 Clave natural de la entidad CARRERA

```
Combinaciones (year, sequence, category) con más de un race_name: 0
```

Conclusión: La tripleta `(year, sequence, category)` identifica de forma unívoca cada carrera en el dataset y constituye su clave primaria natural.

4.6 Dependencia entre equipo y moto

```
Equipos con más de un tipo de moto: 116

Equipo EGO Speed Up      → Boscoscuro, Speed Up
Equipo AGR Team          → Kalex, KTM
Equipo Abbink Bos Racing → Seel, Honda
...
```

Conclusión: `bike` no depende funcionalmente del equipo; depende del contexto concreto de cada resultado. Se ubica en RESULTADO.

4.7 Estabilidad del equipo dentro de una temporada

```
Casos de piloto con más de un equipo en el mismo año y categoría: 329
```

Conclusión de diseño crítica: `id_team` no puede formar parte de la clave primaria de RESULTADO. La relación EQUIPO → RESULTADO es una referencia opcional (FK nullable), no una relación identificadora. Esta decisión previene la fragmentación de puntos al calcular totales por temporada.

5. Verificación de Formas Normales (eda.ipynb — Paso 3)

5.1 Primera Forma Normal (1FN)

Todos los atributos del dataset son atómicos (valores escalares). No existen grupos repetidos ni atributos multivaluados en ninguna fila. **El dataset original cumple la 1FN.**

5.2 Segunda Forma Normal (2FN)

El análisis empírico de dependencias parciales reveló que `category` no actúa como identificador del evento físico:

- `(year, sequence)` → `race_name`, `date` — violación en CARRERA original
- `(year, sequence, rider)` → todos los atributos — violación en RESULTADO

Causa raíz: el modelo original fusionaba dos conceptos distintos en una única entidad CARRERA:

1. El gran premio — evento físico (circuito, fecha, nombre).
2. La prueba competitiva — categoría disputada dentro del evento.

Solución aplicada: separación en `GRAND_PRIX(year, sequence)` y `RACES(year, sequence, category)`. Ambas tablas resultantes cumplen la 2FN.

5.3 Tercera Forma Normal (3FN)

La comprobación exhaustiva de dependencias transitivas mediante permutaciones de atributos no clave en RESULTADO dio el siguiente resultado:

```
Comprobación de Dependencias Transitivas 3FN
Buscando si conocer el valor de un atributo A determina el valor de un atributo B...

CONCLUSIÓN
No se detectaron dependencias transitivas. RESULTADO cumple la 3FN.
```

Para las entidades CIRCUITO y GRAN PREMIO se detectaron dependencias aparentes (`circuit_name → circuit_country`; `date → race_name`), pero se determinó que corresponden a claves candidatas alternativas, no a dependencias transitivas genuinas, por lo que no constituyen violación de la 3FN.

6. Diseño Conceptual

6.1 Diagrama Entidad-Relación

```
CIRCUITS ──────────< GRAND_PRIX >──────────< RACES >──────────< RESULTS
                                                            /        \
                                                        RIDERS       TEAMS
```

Cardinalidades:

Relación	Tipo	Justificación
CIRCUITS → GRAND_PRIX	1:N	Un circuito acoge múltiples GP a lo largo de los años
GRAND_PRIX → RACES	1:N	Un GP engloba varias categorías por fin de semana
RIDERS → RESULTS	1:N	Cada piloto participa en múltiples carreras
RACES → RESULTS	1:N	RESULTS es entidad débil de RACES (relación identificadora)
TEAMS → RESULTS	1:N	FK nullable; 329 casos de cambio de equipo intra-temporada

6.2 Clave primaria de RESULTS

La clave primaria de la tabla RESULTS queda definida como:

```
PK_RESULTS = (id_rider, year, sequence, category)
```

El atributo `id_team` se excluye explícitamente de la clave primaria, quedando como FK nullable. Esta decisión responde a los 329 casos verificados en el §4.7.

7. Modelo Relacional

7.1 Esquema lógico

```
riders (
    id_rider    INT AUTO_INCREMENT  PK,
    forename    VARCHAR(50)         NOT NULL,
    surname     VARCHAR(50)         NOT NULL,
    nationality VARCHAR(50)         NOT NULL
)

teams (
```

```
    id_team     INT AUTO_INCREMENT  PK,
    name        VARCHAR(100)        NOT NULL
)

circuits (
    id_circuit  INT AUTO_INCREMENT  PK,
    name        VARCHAR(100)        NOT NULL,
    country     VARCHAR(50)         NOT NULL
)

grand_prix (
    year        INTEGER             NOT NULL,
    sequence    INTEGER             NOT NULL,
    name        VARCHAR(100)        NOT NULL,
    date        DATE                NOT NULL,
    id_circuit  INTEGER             NOT NULL,
    PRIMARY KEY (year, sequence),
    FK          id_circuit → circuits(id_circuit)
                ON DELETE RESTRICT ON UPDATE CASCADE
)

races (
    year        INTEGER             NOT NULL,
    sequence    INTEGER             NOT NULL,
    category    VARCHAR(20)         NOT NULL,
    PRIMARY KEY (year, sequence, category),
    FK          (year, sequence) → grand_prix(year, sequence)
                ON DELETE RESTRICT ON UPDATE CASCADE
)

results (
    id_rider     INTEGER            NOT NULL,
    year         INTEGER            NOT NULL,
    sequence     INTEGER            NOT NULL,
    category     VARCHAR(20)        NOT NULL,
    id_team      INTEGER            NULL,
    bike         VARCHAR(50),
    position     INTEGER,
    points       DECIMAL(5,1),
    speed        DECIMAL(6,3),
    time         VARCHAR(20),
    rider_number INTEGER            NULL,
    PRIMARY KEY  (id_rider, year, sequence, category),
    FK           id_rider  → riders(id_rider)             ON DELETE RESTRICT ON UPDATE C
    FK           (year, sequence, category) → races       ON DELETE RESTRICT ON UPDATE CA
    FK           id_team   → teams(id_team)               ON DELETE SET NULL ON UPDATE CA
)
```

7.2 Justificación de decisiones de diseño

Atributo	Ubicación final	Justificación
`rider_number`	RESULTS (NULL permitido)	14 conflictos de dorsal; dependencia contextual (año + categoría)

Atributo	Ubicación final	Justificación
`bike`	RESULTS	116 equipos emplean más de una marca de moto
`race_name`	GRAND_PRIX (solo descriptivo)	"Japanese Grand Prix" asociado a dos circuitos distintos
`id_team`	FK nullable en RESULTS	329 casos de piloto con equipos distintos en el mismo año y categoría
`circuit_country`	CIRCUITS	0 circuitos con más de un país tras normalización
`rider_country`	RIDERS	0 pilotos con más de un país registrado
Clave de RACES	`(year, sequence, category)`	Única combinación que identifica una carrera sin ambigüedad
Clave de RESULTS	`(id_rider, year, sequence, category)`	`id_team` excluido de la PK por los 329 casos de cambio intra-temporada

8. Implementación DDL — motogp.sql

El fichero `motogp.sql` crea el esquema `motogp` con codificación `utf8` y cotejamiento `utf8_spanish2_ci`, define las seis tablas en orden de dependencia referencial, e incluye un bloque de vaciado seguro para facilitar la recarga:

```
CREATE SCHEMA IF NOT EXISTS motogp
    DEFAULT CHARACTER SET utf8
    COLLATE utf8_spanish2_ci;

USE motogp;

-- 1. PILOTO
CREATE TABLE riders (
    id_rider    INTEGER AUTO_INCREMENT,
    forename    VARCHAR(50)  NOT NULL,
    surname     VARCHAR(50)  NOT NULL,
    nationality VARCHAR(50)  NOT NULL,
    PRIMARY KEY (id_rider)
);

-- 2. EQUIPO
CREATE TABLE teams (
    id_team INTEGER AUTO_INCREMENT,
    name    VARCHAR(100) NOT NULL,
    PRIMARY KEY (id_team)
);

-- 3. CIRCUITO
CREATE TABLE circuits (
    id_circuit INTEGER AUTO_INCREMENT,
    name       VARCHAR(100) NOT NULL,
    country    VARCHAR(50)  NOT NULL,
    PRIMARY KEY (id_circuit)
);
```

```sql
-- 4. GRAN PREMIO
CREATE TABLE grand_prix (
    year        INTEGER      NOT NULL,
    sequence    INTEGER      NOT NULL,
    name        VARCHAR(100) NOT NULL,
    date        DATE         NOT NULL,
    id_circuit  INTEGER      NOT NULL,
    PRIMARY KEY (year, sequence),
    CONSTRAINT fk_gp_circuits
        FOREIGN KEY (id_circuit) REFERENCES circuits (id_circuit)
        ON DELETE RESTRICT ON UPDATE CASCADE
);

-- 5. CARRERA
CREATE TABLE races (
    year        INTEGER     NOT NULL,
    sequence    INTEGER     NOT NULL,
    category    VARCHAR(20) NOT NULL,
    PRIMARY KEY (year, sequence, category),
    CONSTRAINT fk_race_gp
        FOREIGN KEY (year, sequence) REFERENCES grand_prix (year, sequence)
        ON DELETE RESTRICT ON UPDATE CASCADE
);

-- 6. RESULTADO
CREATE TABLE results (
    id_rider     INTEGER      NOT NULL,
    year         INTEGER      NOT NULL,
    sequence     INTEGER      NOT NULL,
    category     VARCHAR(20)  NOT NULL,
    id_team      INTEGER,
    bike         VARCHAR(50),
    position     INTEGER,
    points       DECIMAL(5,1),
    speed        DECIMAL(6,3),
    time         VARCHAR(20),
    rider_number INTEGER,
    PRIMARY KEY  (id_rider, year, sequence, category),
    CONSTRAINT fk_res_rider
        FOREIGN KEY (id_rider) REFERENCES riders (id_rider)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_res_race
        FOREIGN KEY (year, sequence, category)
        REFERENCES races (year, sequence, category)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_res_team
        FOREIGN KEY (id_team) REFERENCES teams (id_team)
        ON DELETE SET NULL ON UPDATE CASCADE
);

-- VACIADO SEGURO (para recarga sin duplicados)
SET FOREIGN_KEY_CHECKS = 0;
TRUNCATE TABLE results;
TRUNCATE TABLE races;
TRUNCATE TABLE grand_prix;
```

```
TRUNCATE TABLE circuits;
TRUNCATE TABLE teams;
TRUNCATE TABLE riders;
SET FOREIGN_KEY_CHECKS = 1;
```

9. Migración de Datos (eda.ipynb → motogp_mysql.sql)

La migración se realizó en Python mediante `sqlite3` e `INSERT INTO` en bloques de 500 filas. El script exportó el resultado como `motogp_mysql.sql`, que incluye `SET FOREIGN_KEY_CHECKS = 0` al inicio y `= 1` al final para permitir la carga en cualquier orden.

Orden	Tabla	Método de deduplicación	Registros insertados
1	`riders`	`drop_duplicates()` sobre (forename, surname, nationality)	885
2	`teams`	`drop_duplicates()` sobre name (tras sustituir `'?'` por None)	969
3	`circuits`	`drop_duplicates(subset=['circuit_name'])` tras normalización	29
4	`grand_prix`	Agrupado por `(year, sequence)` con `.agg(..., 'first')`	379
5	`races`	`drop_duplicates()` sobre `(year, sequence, category)`	1.121
6	`results`	Sin deduplicación; 0 filas omitidas por error	29.931

9.1 Validación post-migración

```
Conteo final por tabla
  riders        885 filas
  teams         969 filas
  circuits       29 filas
  grand_prix    379 filas
  races       1.121 filas
  results    29.931 filas

Verificación de integridad referencial
  Resultados sin piloto válido:   0
  Resultados sin carrera válida:  0
  Resultados sin equipo (NULL):   5.127  ← esperado: 5.127 ✓
```

10. Consultas SQL — consultas_motogp.sql

Consulta 1 — Campeón de MotoGP en el año más reciente

Obtiene el nombre y apellidos del piloto campeón del mundo en MotoGP para el año más reciente registrado. Se emplea una subconsulta escalar en el `WHERE` para determinar el año máximo disponible, y se filtra por `re.position = 1` para contabilizar únicamente victorias en carrera.

```
-- Subconsulta auxiliar: año más reciente en MotoGP
SELECT MAX(re.year) FROM results re
    INNER JOIN races ra ON ra.year = re.year
        AND ra.sequence = re.sequence
        AND ra.category = re.category
WHERE ra.category = 'MotoGP';

-- Consulta 1
SELECT ri.forename, ri.surname, SUM(re.points) AS puntos_totales, re.position
FROM riders ri
    INNER JOIN results re ON re.id_rider = ri.id_rider
    INNER JOIN races ra   ON ra.year = re.year
        AND ra.sequence = re.sequence
        AND ra.category = re.category
WHERE ra.category = "MotoGP"
  AND re.position = 1
  AND re.year = (
      SELECT MAX(re2.year) FROM results re2
          INNER JOIN races ra2 ON ra2.year = re2.year
              AND ra2.sequence = re2.sequence
              AND ra2.category = re2.category
      WHERE ra2.category = 'MotoGP'
  )
GROUP BY ri.id_rider, ri.forename, ri.surname, re.position
ORDER BY puntos_totales DESC
LIMIT 1;
```

Técnicas empleadas: subconsulta escalar correlacionada en el `WHERE` para parametrizar el filtro temporal; `ORDER BY ... LIMIT 1` como alternativa eficiente cuando se requiere una única fila de resultado.

Consulta 2 — País con más pilotos distintos en la década 2010–2019

Determina el país o países con el mayor número de pilotos diferentes que disputaron al menos una carrera entre 2010 y 2019, excluyendo la categoría MotoE. Se presentan dos implementaciones funcionalmente equivalentes.

```
-- Versión 1: subconsulta anidada en HAVING
SELECT ri.nationality, COUNT(DISTINCT ri.id_rider) AS contador
FROM riders ri
    INNER JOIN results re ON re.id_rider = ri.id_rider
```

```
    INNER JOIN races ra   ON ra.year = re.year
        AND ra.sequence = re.sequence
        AND ra.category = re.category
WHERE ra.category <> "MotoE"
  AND re.year BETWEEN 2010 AND 2019
GROUP BY ri.nationality
HAVING contador = (
    SELECT MAX(T.contador)
    FROM (
        SELECT COUNT(DISTINCT ri2.id_rider) AS contador
        FROM riders ri2
            INNER JOIN results re2 ON re2.id_rider = ri2.id_rider
            INNER JOIN races ra2   ON ra2.year = re2.year
                AND ra2.sequence = re2.sequence
                AND ra2.category = re2.category
        WHERE ra2.category <> 'MotoE'
          AND re2.year BETWEEN 2010 AND 2019
        GROUP BY ri2.nationality
    ) T
);

-- Versión 2 (alternativa con CTE)
WITH conteo AS (
    SELECT ri.nationality AS pais, COUNT(DISTINCT ri.id_rider) AS contador
    FROM riders ri
        INNER JOIN results re ON re.id_rider = ri.id_rider
        INNER JOIN races ra   ON ra.year = re.year
            AND ra.sequence = re.sequence
            AND ra.category = re.category
    WHERE ra.category <> "MotoE"
      AND re.year BETWEEN 2010 AND 2019
    GROUP BY ri.nationality
)
SELECT pais, contador FROM conteo
WHERE contador = (SELECT MAX(contador) FROM conteo);
```

Técnicas empleadas: `COUNT(DISTINCT)` para contabilizar pilotos únicos; `HAVING` con subconsulta de máximo para filtrar el valor más alto; CTE (`WITH`) como construcción que materializa el conjunto de conteos evitando la doble evaluación de la subquery.

Consulta 3 — Pilotos con victorias en MotoGP, Moto2 y Moto3

Identifica los pilotos que han logrado al menos una victoria en carrera (`position = 1`) en cada una de las tres categorías principales. La estrategia consiste en filtrar únicamente los registros de victoria y agrupar contando el número de categorías distintas.

```
SELECT ri.forename, ri.surname,
       COUNT(DISTINCT ra.category) AS num_categorias
FROM riders ri
    INNER JOIN results re ON re.id_rider = ri.id_rider
    INNER JOIN races ra   ON ra.year = re.year
        AND ra.sequence = re.sequence
        AND ra.category = re.category
```

```
WHERE ra.category IN ("MotoGP", "Moto2", "Moto3")
  AND re.position = 1
GROUP BY ri.id_rider, ri.forename, ri.surname
HAVING num_categorias = 3;
```

Técnicas empleadas: `COUNT(DISTINCT ra.category)` como mecanismo de intersección implícita; `HAVING num_categorias = 3` actúa como filtro que exige presencia en los tres conjuntos simultáneamente. Este enfoque resulta más eficiente que tres subqueries independientes con `IN`.

Consulta 4 — Campeones de Moto2 y Moto3 sin título en MotoGP

Obtiene los pilotos que ostentaron el campeonato del mundo en Moto2 y en Moto3, pero no en MotoGP. Se aplica intersección y diferencia de conjuntos sobre `id_rider` mediante los predicados `IN`, `AND IN` y `AND NOT IN`.

```
-- Versión 1: triple subquery con IN / NOT IN
SELECT ri.forename, ri.surname FROM riders ri
WHERE ri.id_rider IN (
    SELECT re.id_rider FROM results re
        INNER JOIN races ra ON ra.year = re.year
            AND ra.sequence = re.sequence AND ra.category = re.category
    WHERE ra.category = "Moto2"
    GROUP BY re.id_rider, ra.year
    HAVING SUM(re.points) = (
        SELECT MAX(puntos_totales) FROM (
            SELECT SUM(re2.points) AS puntos_totales FROM results re2
                INNER JOIN races ra2 ON ra2.year = re2.year
                    AND ra2.sequence = re2.sequence AND ra2.category = re2.category
            WHERE ra2.category = "Moto2" AND ra2.year = ra.year
            GROUP BY re2.id_rider) AS T))
AND ri.id_rider IN (
    SELECT re.id_rider FROM results re
        INNER JOIN races ra ON ra.year = re.year
            AND ra.sequence = re.sequence AND ra.category = re.category
    WHERE ra.category = "Moto3"
    GROUP BY re.id_rider, ra.year
    HAVING SUM(re.points) = (
        SELECT MAX(puntos_totales) FROM (
            SELECT SUM(re2.points) AS puntos_totales FROM results re2
                INNER JOIN races ra2 ON ra2.year = re2.year
                    AND ra2.sequence = re2.sequence AND ra2.category = re2.category
            WHERE ra2.category = "Moto3" AND ra2.year = ra.year
            GROUP BY re2.id_rider) AS T))
AND ri.id_rider NOT IN (
    SELECT re.id_rider FROM results re
        INNER JOIN races ra ON ra.year = re.year
            AND ra.sequence = re.sequence AND ra.category = re.category
    WHERE ra.category = "MotoGP"
    GROUP BY re.id_rider, ra.year
    HAVING SUM(re.points) = (
        SELECT MAX(puntos_totales) FROM (
            SELECT SUM(re2.points) AS puntos_totales FROM results re2
```

```
                INNER JOIN races ra2 ON ra2.year = re2.year
                    AND ra2.sequence = re2.sequence AND ra2.category = re2.category
            WHERE ra2.category = "MotoGP" AND ra2.year = ra.year
            GROUP BY re2.id_rider) AS T));

-- Versión 2 (alternativa con CTE)
WITH puntos_totales AS (
    SELECT re.id_rider, ra.year, ra.category,
           SUM(re.points) AS total_pts
    FROM results re
        INNER JOIN races ra ON ra.year = re.year
            AND ra.sequence = re.sequence AND ra.category = re.category
    GROUP BY re.id_rider, ra.year, ra.category
),
campeones AS (
    SELECT pt.id_rider, pt.year, pt.category
    FROM puntos_totales pt
    WHERE pt.total_pts = (
        SELECT MAX(pt2.total_pts) FROM puntos_totales pt2
        WHERE pt2.year = pt.year AND pt2.category = pt.category)
)
SELECT DISTINCT ri.forename, ri.surname
FROM riders ri
WHERE ri.id_rider IN     (SELECT id_rider FROM campeones WHERE category = 'Moto2')
  AND ri.id_rider IN     (SELECT id_rider FROM campeones WHERE category = 'Moto3')
  AND ri.id_rider NOT IN (SELECT id_rider FROM campeones WHERE category = 'MotoGP');
```

Resultado obtenido: Álex Márquez — campeón de Moto3 en 2014 y de Moto2 en 2019, sin título en MotoGP en el período 2000–2021.

Consulta 5 — Equipos con más campeonatos del mundo en MotoGP

Determina los equipos ordenados por el número de temporadas en las que alguno de sus pilotos fue campeón del mundo. La asignación del equipo a cada campeonato se realiza mediante una subconsulta escalar con `ORDER BY SUM(points) DESC LIMIT 1`, que selecciona el equipo con el que el campeón acumuló más puntos ese año.

Los campeonatos de 2002, 2003 y 2004 (Valentino Rossi) quedan excluidos automáticamente por el `INNER JOIN` con `teams`, dado que sus registros presentan `id_team = NULL` como consecuencia del token `'?'` en el dataset original.

```
-- Subconsulta auxiliar: campeón de cada año en MotoGP
SELECT re.id_rider, ri.forename, ri.surname, ra.year FROM results re
    INNER JOIN riders ri ON ri.id_rider = re.id_rider
    INNER JOIN races ra  ON ra.year = re.year
        AND ra.sequence = re.sequence AND ra.category = re.category
WHERE ra.category = 'MotoGP'
GROUP BY re.id_rider, ra.year
HAVING SUM(re.points) = (
    SELECT MAX(puntos_totales) FROM (
        SELECT SUM(re2.points) AS puntos_totales FROM results re2
            INNER JOIN races ra2 ON ra2.year = re2.year
```

```
                AND ra2.sequence = re2.sequence AND ra2.category = re2.category
        WHERE ra2.category = 'MotoGP' AND ra2.year = ra.year
        GROUP BY re2.id_rider) AS T);

-- Consulta 5
SELECT te.name, COUNT(*) AS num_campeones
FROM teams te
INNER JOIN (
    SELECT re.id_rider, ra.year,
        (SELECT re2.id_team FROM results re2
             INNER JOIN races ra2 ON ra2.year = re2.year
                 AND ra2.sequence = re2.sequence AND ra2.category = re2.category
         WHERE ra2.category = "MotoGP"
           AND re2.id_rider = re.id_rider
           AND ra2.year = ra.year
         GROUP BY re2.id_team
         ORDER BY SUM(re2.points) DESC
         LIMIT 1) AS E
    FROM results re
        INNER JOIN races ra ON ra.year = re.year
            AND ra.sequence = re.sequence AND ra.category = re.category
    WHERE ra.category = 'MotoGP'
    GROUP BY re.id_rider, ra.year
    HAVING SUM(re.points) = (
        SELECT MAX(puntos_totales) FROM (
            SELECT SUM(re2.points) AS puntos_totales FROM results re2
                INNER JOIN races ra2 ON ra2.year = re2.year
                    AND ra2.sequence = re2.sequence AND ra2.category = re2.category
            WHERE ra2.category = 'MotoGP' AND ra2.year = ra.year
            GROUP BY re2.id_rider) AS T)
) AS C ON te.id_team = C.E
GROUP BY te.id_team, te.name
ORDER BY num_campeones DESC;
```

Resultado obtenido: Repsol Honda Team encabeza la clasificación. Los 3 campeonatos del período 2002–2004 quedan excluidos por ausencia de equipo identificado en el dataset.

Consulta 6 — Circuitos sin ganador local en ninguna categoría

Obtiene los circuitos en los que ningún piloto cuya nacionalidad coincida con el país del trazado haya ganado una carrera en ninguna de las categorías registradas. Se aplica el patrón de **diferencia de conjuntos** mediante `NOT IN`. La correcta resolución de esta consulta fue posible gracias a la homogeneización de códigos de país documentada en el §3.4.

```
-- Subconsulta auxiliar: circuitos con al menos una victoria local
SELECT DISTINCT gp.id_circuit FROM grand_prix gp
    INNER JOIN races ra    ON ra.year = gp.year AND ra.sequence = gp.sequence
    INNER JOIN results re  ON re.year = ra.year
        AND re.sequence = ra.sequence AND re.category = ra.category
    INNER JOIN riders ri   ON ri.id_rider = re.id_rider
    INNER JOIN circuits ci ON ci.id_circuit = gp.id_circuit
WHERE re.position = 1
  AND ri.nationality = ci.country;
```

```sql
-- Consulta 6
SELECT ci.name, ci.country FROM circuits ci
WHERE ci.id_circuit NOT IN (
    SELECT DISTINCT gp.id_circuit FROM grand_prix gp
        INNER JOIN races ra     ON ra.year = gp.year AND ra.sequence = gp.sequence
        INNER JOIN results re   ON re.year = ra.year
            AND re.sequence = ra.sequence AND re.category = ra.category
        INNER JOIN riders ri    ON ri.id_rider = re.id_rider
        INNER JOIN circuits ci2 ON ci2.id_circuit = gp.id_circuit
    WHERE re.position = 1
      AND ri.nationality = ci2.country)
ORDER BY ci.country, ci.name;
```

Resultado obtenido: 14 circuitos sin victoria de piloto local en ninguna categoría:

Circuito	País
Termas De Río Hondo	ARG
Red Bull Ring - Spielberg	AUT
Nelson Piquet Circuit	BRA
Shanghai Circuit	CHN
Automotodrom Brno	CZE
Sepang International Circuit	MAL
TT Circuit Assen	NED
Estoril Circuit	POR
Losail International Circuit	QAT
Phakisa Freeway	RSA
Chang International Circuit	THA
Istanbul Circuit	TUR
Circuit Of The Americas	USA
Indianapolis Motor Speedway	USA

Los dos circuitos estadounidenses ilustran el alcance preciso de la consulta: la existencia de un campeón del mundo americano (Nicky Hayden, 2006) no implica que haya ganado una carrera en dichos trazados específicamente.

11. Conclusiones

El proceso de diseño y migración realizado valida empíricamente los principios fundamentales de la normalización relacional y pone de manifiesto la importancia de una auditoría rigurosa previa al diseño.

El análisis de datos es previo e indispensable al diseño. Sin la identificación de los 14 conflictos de dorsal, los 329 cambios de equipo intra-temporada, la ambigüedad de `race_name` o la inconsistencia de códigos de país, el modelo resultante habría incorporado claves incorrectas o producido resultados erróneos en las consultas analíticas.

La evidencia cuantitativa fundamenta las decisiones de diseño. La exclusión de `id_team` de la clave primaria de RESULTS y la ubicación de `bike` en dicha tabla no responden a convenciones genéricas, sino a hechos medibles extraídos del dataset (116 equipos con más de una moto; 329 cambios de equipo intra-temporada).

La granularidad del `GROUP BY` determina la corrección de las consultas agregadas. Durante el desarrollo de la Consulta 5 se constató que una granularidad incorrecta en el `GROUP BY` exterior podía excluir campeonatos válidos. El nivel de agrupación de la consulta exterior debe ser siempre igual o más general que el de la subquery de cálculo del máximo.

La homogeneización semántica forma parte del proceso de diseño. La inconsistencia entre `riders.nationality` (alpha-3) y `circuits.country` (alpha-2) no era detectable en el DDL, pero impedía la resolución correcta de la Consulta 6. Su corrección mediante un `UPDATE` documentado en `motogp.sql` constituye una decisión de diseño con impacto directo en la integridad semántica de las relaciones entre tablas.

La normalización a 3FN elimina anomalías operacionales reales. En el esquema final, la actualización del nombre de un circuito requiere modificar una única fila en la tabla `circuits`; en el esquema original, la misma operación habría exigido actualizar cientos de registros con riesgo de inconsistencia.

Documento generado el 04/04/2026 — Bases de Datos I, Curso 2025/2026